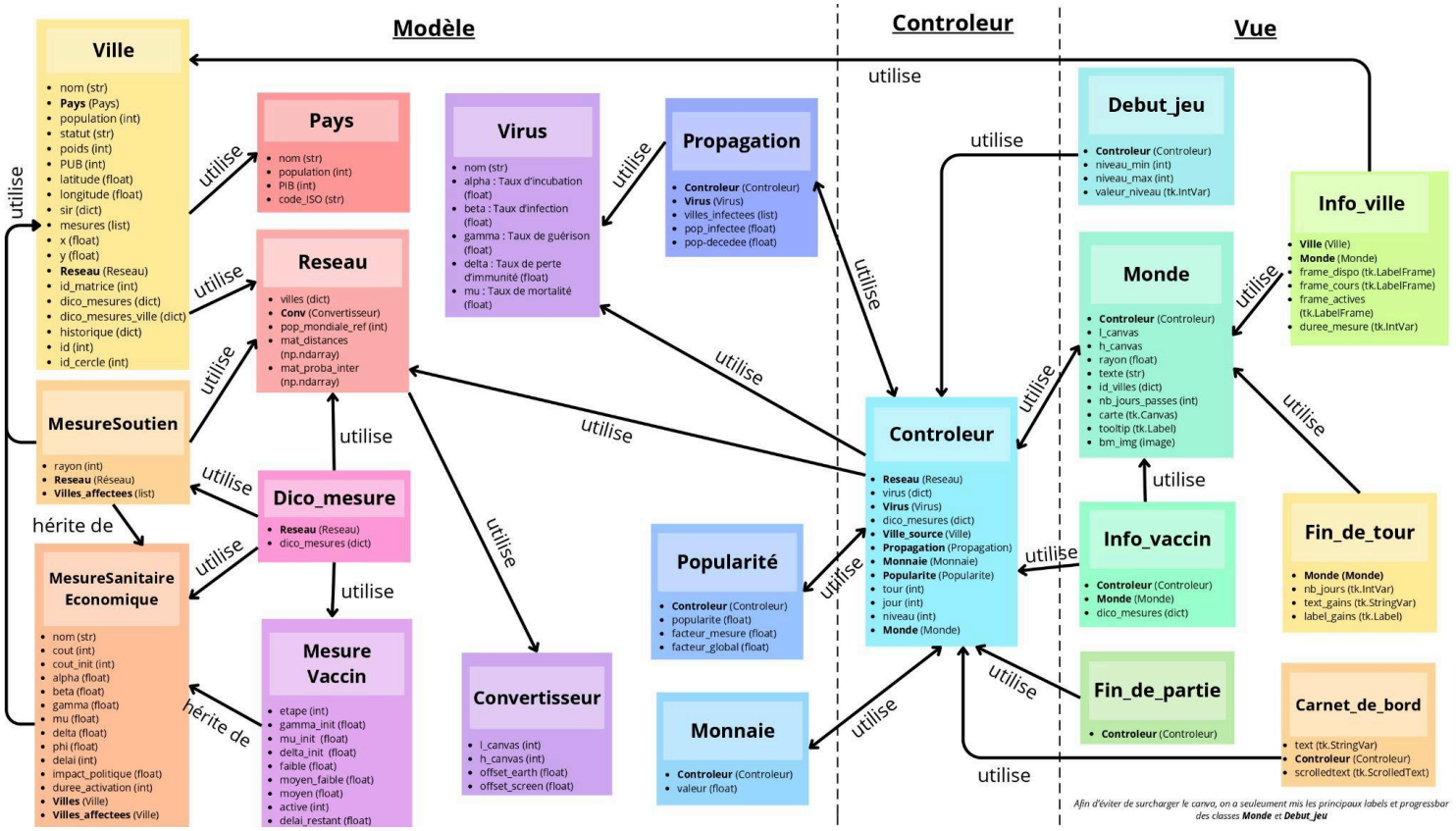
Rapport de projet

1. Découpage fonctionnel

1.1 Découpage initial (réalisé à l'étape 2) :

Visible en **Annexe**

1.2 Découpage final :



1.3 Justification des différences :

- Différences dans le Modèle :

- **Ville** s'enrichit : les coordonnées sont séparées (géographiques / canvas), et la ville stocke maintenant les identifiants associés à son affichage, son historique SEIRD et ses mesures locales
- **Réseau** : la matrice_adj est remplacée par 2 matrices plus adaptées : mat_distances et mat_proba_inter
- **Virus :** ajout d'un nom nécessaire pour l'affichage et la sélection par le joueur

Ajout de nouvelles classes :

- **Dico_mesures** : permet de gérer toute la mise en place des mesures sanitaires
- **MesureSanitaireEconomique, MesureSoutien, MesureVaccin** : permet de modéliser différents types de mesures avec héritage
- **Propagation, Monnaie, Popularité :** pour alléger le contrôleur

Lors de l'étape 2, nous n'avions pas arrêté notre réflexion sur le fonctionnement précis des mesures, et nous en avions fortement sous-estimé la complexité, ce qui justifie le besoin de ces ajouts.

- Différences dans le Contrôleur :

Nous avons cherché à épurer le contrôleur en déléguant un maximum à de nouvelles classes (**Propagation, Monnaie, Popularité**) que l'on appellera depuis le contrôleur.

- avancee_vaccin, popularite, fonds, dico_villes ont été supprimés et déléguées à des classes dédiées (MesureVaccin, Popularite, Monnaie, …)

- Différences dans la Vue :

- **Monde** et **Info_ville** s'enrichissent : suite au design de l'UI, il possède des attributs d'affichage plus poussée (rayon, texte, id_villes, nb_jours_passes, tooltip, bm_img, frames, graphiques…)
- **Monde** : la vue ne stocke plus d'information, et passe bien par le contrôleur, nous permettant de supprimer dico_villes

Ajout de nouvelles fenêtres suite à l'implémentation / la gestion de nouvelles fonctionnalités :

- **Debut_jeu** : pour sélectionner le niveau / virus
- **Info_vaccin** : dédiée à la recherche vaccinale
- **Fin_de_tour** et **Fin_de_partie** : participent à la gestion fluide des transitions de jeu

Résumé :

Le nouveau découpage est plus cohérent vis-à -vis des principes MVC. Le modèle s'est enrichi en classes spécialisées (Dico_mesures, MesureSanitaireEconomique, MesureVaccin…). Le contrôleur a été allégé en délégant vers de nouvelles classes du Modèle (Popularité, Propagation et Monnaie). La vue s'est complexifiée pour nous permettre d'avoir une interface plus intéractive et complète. De plus, maintenant, elle va chercher directement les informations dans le contrôleur au lieu de les stocker.

2. Analyse de Complexité et Optimisation

L'algorithme de propagation constitue le cœur calculatoire de notre modèle épidémiologique. Il simule à la fois l'évolution interne de la maladie au sein des populations urbaines (modèle compartimental SEIRD) via la méthode **propagation_intra**, et la diffusion du virus à l'échelle du réseau mondial avec **propagation_inter**. Cette section détaille les structures de données employées, l'estimation de la complexité de l'algorithme non-trivial, et les compromis d'optimisation entre les deux versions du programme.

2.1 Structures de données, modélisation du réseau et de la propagation

Le réseau mondial est modélisé sous la forme d'un graphe complet implicite, stocké via un dictionnaire Python associant le nom de chaque ville à son objet **Ville**. Afin de traiter efficacement la propagation inter-villes, nous avons fait le choix de pré-calculer les interactions géographiques à l'aide de tableaux vectorisés.

Ces intéractions géographiques passent ainsi par une matrice d'adjacence et une matrice des probabilités inspirée du modèle gravitaire. Plutôt que de recalculer les distances à la volée, une matrice 2D NumPy des distances est générée à l'initialisation du réseau. Celle-ci est complétée par une matrice des probabilités normalisées permettant que la somme des probabilités de propagation sortantes d'une ville valle toujours 1, stabilisant le modèle quel que soit le nombre de villes. Lors de la création du graphe, chaque ville se voit attribuer un attribut **id_matrice** (entier de **0** à **V-1**) qui permet de faire le lien entre l'objet et la matrice avec un coût en **O(1)**.

Sur le plan de la dynamique épidémiologique, l'algorithme applique la méthode **propagation_intra** sur l'intégralité de la durée de propagation choisie par le joueur avant d'appeler **propagation_inter**, indépendamment de la durée choisie, et avec une virulence dépendant de l'avancée de la contamination dans les villes infectées. En permettant ce découplage, le modèle introduit une mécanique essentielle de prise de risque / récompense car le gain monétaire est également non-linéaire vis-à-vis de la durée de propagation.
Ce choix permet par ailleurs de décroitre la complexité car **propagation_inter** n'est appelée qu'une fois quel que soit le nombre de jours de propagation simulés.

2.2 Évolution de l'Algorithme : Version 1 vs Version 2

A) Propagation Inter-Villes

Cette phase s'appuie sur une double boucle parcourant les villes actuellement infectées (**I**) et confrontant chacune d'elles au reste du réseau (**V**).

Dans la Version 1, à chaque itération, le programme recrée la liste des villes et utilise la méthode **.index(ville)** pour retrouver la position de la ville source et de la ville cible dans la matrice, avec un coût de **O(V)**. La complexité de cette phase atteint donc **O(I * V²)**, et

l'algorithme devient cubique **O(V³)** dans le pire cas, lorsque la majorité des villes sont infectées (**I** ≈ **V**).

Dans la Version 2, le contournement de la méthode **.index()** grâce à l'attribut **id_matrice** calculé à l'initialisation permet un accès direct à la matrice. Le coût de la recherche chute à **O(1)**. La complexité de la double boucle est ainsi réduite à **O(I * V),** ou **O(V²)** dans le pire cas.

B) Propagation Intra-Ville

L'intégration numérique des équations différentielles repose sur la méthode d'Euler explicite, itérant sur le nombre de pas de temps **N = n/dt** (avec **n** le nombre de jours).

Dans la Version 1, l'algorithme manipule des populations et crée de nouveaux objets *numpy.array* à chaque itération de la boucle d'Euler. Sur de grands, l'apport massif d'infections externes provoque des dépassements de capacité mathématiques (*Overflow*), entraînant la génération de valeurs aberrantes (*NaN*).

Dans la Version 2, nous avons abandonné les tableaux NumPy à l'intérieur de la boucle au profit de variables natives Python, éliminant l'important surcoût de création d'objets dans des boucles de milliers d'itérations.

Nous avons également mis en place une normalisation des variables SEIRD en proportions (de 0 à 1) plutôt qu'en effectifs absolus, permettant d'effectuer les calculs avec des valeurs de l'ordre de 1 plutôt que du million. Cette normalisation est couplée à un encadrement des valeurs issues de l'intégration numérique entre 0 et 1. Cela fiabilise intégralement le comportement de la méthode d'Euler contre les erreurs de dépassement des flottants, garantissant un schéma de calcul stable en **O(1)** pour chaque pas de temps.
Bien que cela ne réduise pas la complexité, les calculs sont légèrement plus rapides à chaque itération, ce qui se caractérise par un gain de temps non négligeable au global.

Cependant, sécuriser la méthode d'Euler implique l'appel aux fonctions **min** et **max** à chaque pas de temps, pour chaque compartiment, générant un léger surcoût d'exécution, mais cet arbitrage technique est nécessaire car il permet de garantir une stabilité numérique absolue à la simulation, là où la Version 1 finissait par crasher.

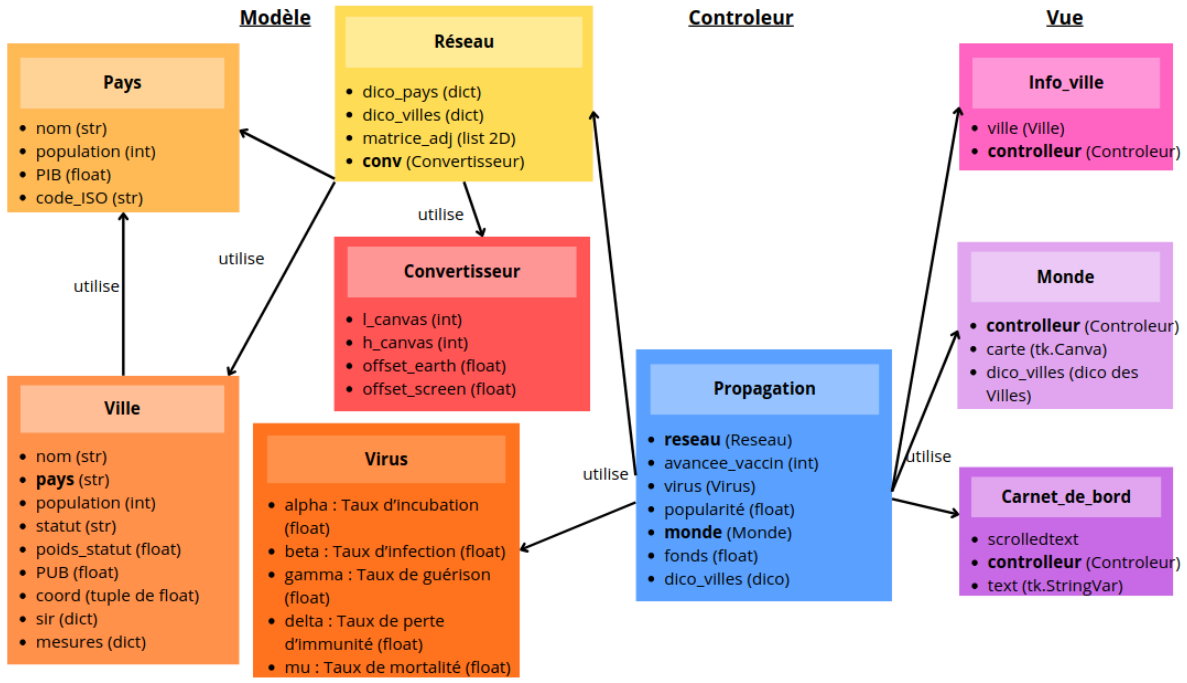
C) Synthèse de la complexité globale de la méthode de propagation :

Soit n le nombre de jours simulés, dt le pas de temps du schéma d'Euler, **V** le nombre total de villes, et **I** le nombre de villes infectées (1 < **I** < **V**).
La complexité temporelle globale de la Version 2 s'établit à : **O(I * n/dt + I * V)**.
Étant donné que dt est une constante (0.01), l'expression se simplifie en **O(I * n + I * V)**.

Des relevés expérimentaux ont été conduits à l'aide d'un script de benchmark automatisé (**calculs_complexite.py**), validant l'analyse théorique. Ils sont disponibles en annexe.

3. Annexes

Annexe 1 : Découpage initial



Annexe 2 : Validation Expérimentale des Complexités

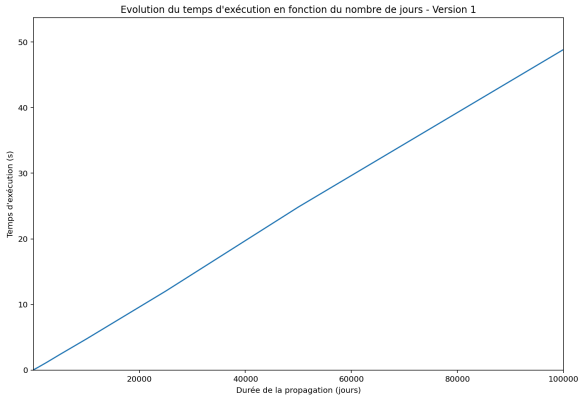
Afin d'isoler et de mesurer précisément la complexité temporelle de la dynamique épidémiologique, le script **calculs_complexite.py** utilise des versions épurées et simplifiées de certaines classes. Cette simplification volontaire permet de se concentrer exclusivement sur la partie “non triviale” de notre algorithme, garantissant des mesures de temps d'exécution stables, fidèles à la seule complexité algorithmique sous-jacente, même lors de simulations intensives sur un grand nombre de jours ou de villes.

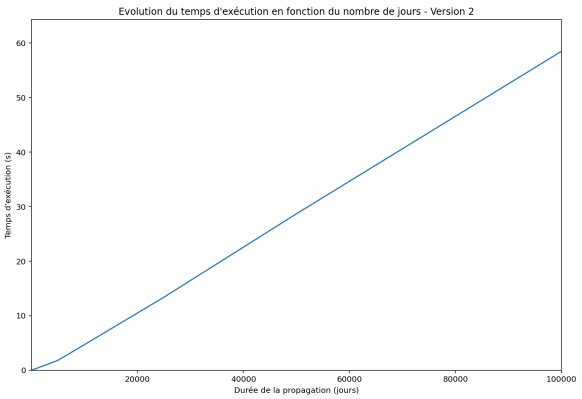
Évolution du temps d'exécution en fonction du nombre de villes **V** (Grippe, **n** = 100) :

Notre algorithme n’ayant pas été pensé pour des simulation sur de réseaux massifs, la propagation est incohérente pour des réseau de plusieurs centaines / milliers de noeuds quelle que soit la version :

- Avec des distances trop faibles, la version 1 ne conserve pas les populations et les calculs explosent
- Avec la matrice de probabilité inspirée du modèle gravitaire, les probabilités de propagation entre les villes deviennent négligeables.

Évolution du temps d'exécution en fonction du nombre de jours **n** (Grippe, **V** = 100):

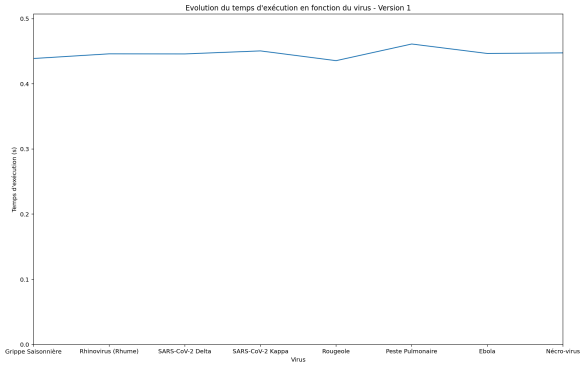


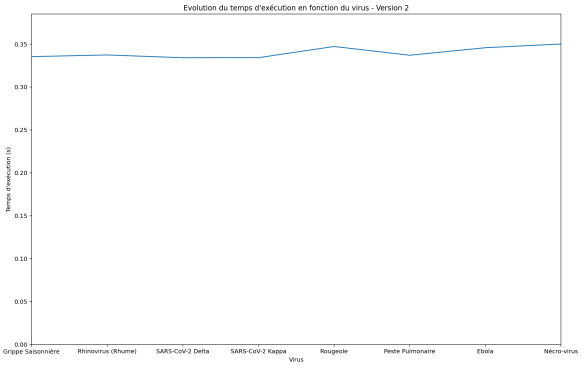


Les courbes sont des droites strictement linéaires pour les deux versions. On note que la Version 2 est environ 20 % plus lente que la Version 1 sur cet axe spécifique.

Explication algorithmique : La méthode d'Euler itère sur le pas de temps **dt**. Le temps de calcul est donc strictement proportionnel à **n**, validant la composante **O(l * n)** de notre formule. Le léger retard de la Version 2 illustre le coût du traitement conditionnel des barrières mathématiques (**min**/**max**) requises pour empêcher les instabilités numériques.

Évolution du temps d'exécution en fonction du type de virus (**n** = 1000, **V** = 100) :





Dans les deux versions, le temps de calcul est le même pour tous les virus, au facteur aléatoire près. On remarque que la Version 1 est environ 30 % plus lente que la Version 2.

Explication algorithmique : Une fois que le virus a contaminé toutes les villes du réseau (très rapide pour 100 villes par rapport aux 1000 jours de simulation), on a alors **I** ≈ **V** et tous les virus nécessitent le même temps de calcul. Si la simulation avait été effectuée sur une période beaucoup plus courte, les virus dont la contamination à travers le monde est plus rapide auraient atteint des valeurs de **I** plus élevées et on aurait pu observer une différence entre eux.